

Nguyên lý thiết kế kiến trúc (Architecture Design Principles)

PHỤ THUỘC B/04-nen-tang-hoc-tu-du-lieu (*inductive bias* và trục “giả định mạnh + ít dữ liệu ↔ giả định yếu + nhiều dữ liệu” — chương này chính là trục đó áp cho kiến trúc); B/09-thiet-ke-ham-mat-mat (chỗ cài giả định thứ hai); A/01-toan-toi-thieu/04-hinh-hoc-va-tin-hieu (convolution, lấy mẫu).

TÓM TẮT

Chương này giải thích vì sao không thể dùng một MLP khổng lồ cho mọi thứ, và vì sao mỗi kiến trúc thực chất là một tập giả định về cấu trúc dữ liệu được đông cứng thành mã nguồn. Nó xây năm câu hỏi để mổ xẻ một kiến trúc lạ, kèm phép tính tay ra con số cho từng câu: giả định nào được cài vào, mô hình nhìn được bao xa, thông tin định tuyến cố định hay theo nội dung, gradient chảy thế nào, và bốn thước đo chi phí rơi vào đâu. Đọc xong, bạn dự đoán được một lớp lỗi trước khi huấn luyện, tức trường tiếp nhận không phủ hết vật thể. Bạn giải thích được vì sao thêm độ sâu và thêm độ rộng có giá hoàn toàn khác nhau. Và bạn nhận ra khi nào một bảng so sánh kiến trúc thực ra đang so sánh công thức huấn luyện. Nếu chỉ nhớ một điều: kiến trúc không làm mô hình “thông minh hơn”, nó quyết định *ai chạm được tới ai, sau bao nhiêu bước, và với chi phí nào*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tensor và feature map** — tensor là mảng số nhiều chiều — ảnh đầu vào là tensor $C \times H \times W$ (kênh, cao, rộng). Feature map là tensor ở giữa mạng: mỗi “kênh” của nó là một bản đồ cho biết một loại đặc trưng xuất hiện mạnh ở vị trí nào.
- **tầng nối đầy đủ và convolution** — tầng nối đầy đủ (*fully connected*) nhân toàn bộ đầu vào với một ma trận — mọi đầu ra nhìn thấy mọi đầu vào. Convolution trượt một cửa sổ nhỏ (kernel) khắp ảnh và dùng *cùng* bộ trọng số ở mọi vị trí, nên mỗi đầu ra chỉ nhìn thấy một vùng nhỏ.
- **kernel, stride, padding** — kernel là kích thước cửa sổ trượt (thường 3×3); stride là bước nhảy giữa hai vị trí liên tiếp (stride 2 làm ảnh nhỏ đi một nửa); padding là số pixel đệm thêm ở viền để kiểm soát kích thước đầu ra.
- **hàm kích hoạt (activation function)** — hàm áp lên từng phần tử giữa hai tầng tuyến tính, ví dụ ReLU $\max(0, x)$. Không có nó thì xếp chồng bao nhiêu tầng tuyến tính cũng chỉ tương đương một tầng tuyến tính — đó là lý do nó bắt buộc phải có.
- **lan truyền ngược và luồng gradient** — để cập nhật trọng số ở tầng đầu, đạo hàm phải nhân dồn ngược qua tất cả các tầng phía sau. Chuỗi phép nhân dài đó có thể tiêu biến hoặc bùng nổ, và đó là lý do “đường về của gradient” là một tiêu chí thiết kế thật sự chứ không phải ẩn dụ.
- **tham số với FLOPs** — tham số là số trọng số phải lưu; FLOPs là số phép nhân-cộng phải làm cho một lần chạy. Hai đại lượng độc lập với nhau: convolution có ít tham số nhưng nhiều FLOPs vì cùng bộ trọng số được dùng lại ở mọi vị trí.
- **inductive bias** — tập giả định mà bạn cài sẵn vào hệ thống về hình dạng của lời giải, trước khi nhìn dữ liệu. Nó đến từ bốn chỗ: kiến trúc, hàm mất mát, augmentation, và thuật toán tối ưu. Bạn không thể “không có giả định” — chỉ có thể chọn giả định nào.
- **batch** — số mẫu được xử lý cùng lúc trong một bước cập nhật. Nó ảnh hưởng tới bộ nhớ, tới độ ổn định của gradient, và — như phần cuối chương cho thấy — tới cả độ trễ đo được.

CÂU HỎI MỞ ĐẦU

- Q1. Một MLP đủ rộng xấp xỉ được mọi hàm liên tục — vậy vì sao không một hệ thống thị giác nghiêm túc nào dùng MLP cho ảnh?
- Q2. Trước ResNet, mạng 56 tầng có sai số *huấn luyện* cao hơn mạng 20 tầng. Vì sao đó chắc chắn không phải *overfitting*, và cái gì thực sự hỏng?
- Q3. Nếu *attention* mạnh hơn convolution ở mọi mặt lý thuyết, vì sao thể hệ ViT tiếp theo lại phải cắt nó về thành cục bộ trong từng cửa sổ?
- Q4. Hai mạng cùng số tham số, một sâu gấp đôi và một rộng gấp đôi — vì sao cái rẻ hơn về tham số lại là cái đắt hơn về thời gian chạy?
- Q5. Bạn đọc một bảng nói kiến trúc mới hơn kiến trúc cũ 3 điểm mAP. Vì sao con số đó có thể không nói gì về kiến trúc cả?

1.1 Đặt vấn đề

Một mạng nơ-ron đủ rộng xấp xỉ được mọi hàm liên tục; định lý xấp xỉ phổ quát nói vậy từ những năm 1980. Nếu thế thì vì sao không dùng một **MLP** khổng lồ cho mọi thứ và để dữ liệu lo phần còn lại? Câu trả lời không nằm ở khả năng biểu diễn mà nằm ở *chi phí học*: định lý nói hàm đúng *tồn tại* trong không gian giả thuyết, nó không nói bạn tìm được hàm đó với lượng dữ liệu và lượng tính toán bạn có. Kiến trúc là công cụ thu hẹp không gian giả thuyết lại quanh những hàm khả dĩ, và mọi lần thu hẹp đều là một phát biểu về cấu trúc của dữ liệu.

Ba ví dụ để thấy phát biểu đó cụ thể tới mức nào:

- **Convolution** phát biểu rằng đặc trưng hữu ích thì *cục bộ*, và một đặc trưng có nghĩa ở góc trên trái thì cũng có nghĩa y như vậy ở góc dưới phải.
- **Pooling** phát biểu rằng vị trí chính xác ít quan trọng hơn sự hiện diện.
- **Attention** phát biểu rằng quan hệ giữa hai vị trí nên do *nội dung* quyết định chứ không do khoảng cách.

Nếu phát biểu khớp với dữ liệu, bạn được tổng quát hoá tốt với ít mẫu; nếu không khớp, bạn đã cài sẵn một sai lệch mà không lượng dữ liệu nào gỡ được. Đây chính xác là trục ở chương 4: giả định mạnh đối lấy ít dữ liệu. Toàn bộ lịch sử kiến trúc thị giác đọc được như một chuỗi các lần *nói lòng* giả định, mỗi lần dữ liệu và tính toán rẽ đi một bậc.

↪ **tiền đề** B/04 — trục đánh đổi giả định–dữ liệu là thứ giải thích vì sao ViT cần tiền huấn luyện quy mô lớn còn CNN thì không.

Giá trị của chương: sau khi đọc, bạn không cần nhớ tên kiến trúc nào cũng đọc được một bài báo kiến trúc lạ, vì bạn có năm câu hỏi cố định và ba phép tính tay để tự trả lời chúng.

TRỰC GIÁC

Hãy nghĩ về kiến trúc như hệ thống ống dẫn chứ không như một cái hộp đen: nó quyết định *thông tin nào có thể chạm tới thông tin nào, sau bao nhiêu bước*. Một MLP là bể chứa — mọi thứ chạm được mọi thứ ngay lập tức, nên nó không có cấu trúc để dựa vào và phải trả bằng tham số. Convolution là mạng ống ngăn nối các điểm lân cận, nên hai pixel ở hai góc ảnh chỉ “nói chuyện” được sau nhiều tầng. Nếu số tầng không đủ, chúng *không bao giờ* nói chuyện được, dù bạn huấn luyện bao lâu. **Kết nối tắt** (*residual connection*) là đường ống phụ chạy song song, đặt ở đó không phải để mang thêm thông tin xuôi mà để **gradient** có lối về ngấn. Đọc kiến trúc theo hình dạng đường ống thì receptive field, attention và skip

connection thôi là ba khái niệm rời rạc và trở thành ba câu trả lời cho cùng một câu hỏi: ai chạm được tới ai.

1.2 Từ template matching tới convolution (From Template Matching to Convolution)

A. Điểm xuất phát nhìn được bằng mắt

Điểm xuất phát đáng ngạc nhiên là bộ phân loại tuyến tính $f = Wx + b$. Cách đọc bề bợt không phải đại số mà là hình học: mỗi hàng của W là một **mẫu chuẩn** (*template*) có cùng số chiều với ảnh, và điểm số của một lớp là tích vô hướng giữa ảnh và mẫu chuẩn đó. Ý nghĩa bằng lời: *mô hình chấm điểm bằng cách hỏi ảnh này giống bản mẫu của lớp đến đâu*. Vẽ các hàng của W ra thành ảnh sau khi huấn luyện trên CIFAR-10 là bài tập rất đáng làm. Bạn sẽ thấy mẫu “ngựa” có hai đầu. Lý do là mỗi lớp chỉ có *một* mẫu chuẩn, nên nó buộc phải trung bình mọi tư thế lại. Giới hạn của mô hình lộ ra từ hình ảnh đó, trước cả khi bạn đọc con số accuracy.

B. Ba lần thu hẹp không gian giả thuyết

Mạch 1 — vì sao convolution ra đời, và nó lấy đi cái gì

- 1 Vấn đề** một mẫu chuẩn cho mỗi lớp là quá ít, và biên quyết định chỉ là siêu phẳng.

Giải pháp xếp chồng nhiều tầng tuyến tính xen phi tuyến, tức MLP.

Giả định hàm cần học liên tục và mạng đủ rộng.

Giá phải trả số tham số nổ tung — ảnh $224 \times 224 \times 3 = 150\,528$ chiều nối đầy đủ vào 1000 nơ-ron là ≈ 150 triệu tham số cho *một* tầng.

Vấn đề mới tệ hơn cả chi phí, mô hình không hề biết hai pixel cạnh nhau thì liên quan, nên phải học lại sự thật đó từ dữ liệu, và học lại nó riêng cho từng vị trí.
- 2 Vấn đề** quá nhiều tham số và không có chút tri thức nào về ảnh.

Giải pháp **chia sẻ trọng số** (*weight sharing*) cộng tính cục bộ, tức convolution [1].

Hệ quả ta được **đẳng biến tịnh tiến** (*translation equivariance*) gần như miễn phí, và số tham số giảm hàng nghìn lần: một tầng conv 3×3 từ 3 kênh sang 64 kênh chỉ có $3 \cdot 3 \cdot 3 \cdot 64 = 1728$ trọng số, *bất kể ảnh to bao nhiêu*.

Giá phải trả bất biến với xoay và với tỉ lệ thì *không* được cài vào, nên mạng vẫn phải học chúng từ dữ liệu hoặc từ **augmentation** — tức chỗ cài giả định thứ ba của chương 4; và quan hệ giữa hai vùng xa nhau trở nên đắt đỏ.

Vấn đề mới nếu thông tin chỉ lan ra từng chút qua mỗi tầng thì *mô hình nhìn được bao xa?*

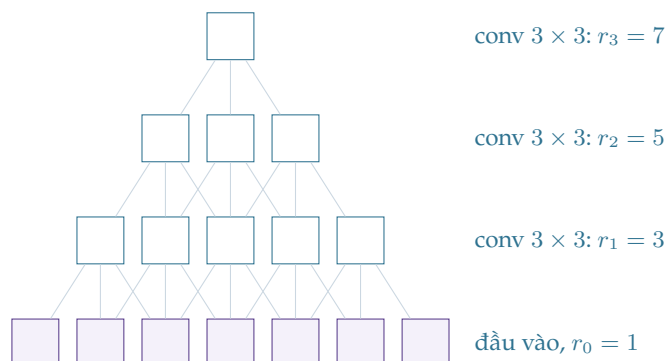
Giải pháp tính receptive field bằng tay *trước* khi huấn luyện, thay vì phát hiện vấn đề sau ba ngày.

C. Receptive field: con số tính được trước khi train

Kích thước đầu ra của một tầng conv là $(W - F + 2P)/S + 1$, và **trường tiếp nhận** (*receptive field*) tích lũy theo hai công thức truy hồi rất đơn giản:

$$j_l = j_{l-1} \cdot s_l, \quad r_l = r_{l-1} + (k_l - 1) j_{l-1}, \quad j_0 = 1, r_0 = 1.$$

Diễn giải bằng lời: mỗi tầng mở rộng tầm nhìn thêm đúng $(k - 1)$ lần bước nhảy hiện tại, và mỗi lần lấy mẫu xuống thì bước nhảy nhân lên. Hệ quả là stride mở rộng tầm nhìn theo cấp số nhân, còn độ sâu chỉ mở rộng theo cấp số cộng. Hình 1.1 minh hoạ trường hợp nhỏ nhất tính được trong đầu.



Hình 1.1. Ba tầng 3×3 stride 1 (vẽ theo một chiều cho gọn): một nơ-ron ở tầng trên cùng nhìn thấy đúng 7 pixel đầu vào, tô đậm. Cùng tầm nhìn với một tầng 7×7 nhưng chỉ tốn $3 \cdot 9C^2 = 27C^2$ tham số thay vì $49C^2$, và có ba phi tuyến thay vì một — đây chính là lập luận của VGG.

Vì sao con số này quan trọng đến thế? Vì *receptive field* nhỏ hơn vật thể là nguyên nhân của một lớp lỗi rất khó chẩn đoán. Mô hình sẽ sai có hệ thống trên vật lớn, và trên những gì cần ngữ cảnh rộng: phân biệt một con thuyền với một cái xe đời mới nhìn thấy nước hay đường. Triệu chứng thì trông giống hệt “thiếu dữ liệu” hoặc “mô hình nhỏ quá”, và cả hai chẩn đoán đó đều dẫn tới hành động sai.

↔ **đối lập với** B/08 — cùng triệu chứng “sai có hệ thống trên một nhóm”, nhưng ở đó nguyên nhân là dịch chuyển phân phối chứ không phải giới hạn kiến trúc; phép đo phân biệt hai thứ là khác nhau.

Điều dễ chịu là receptive field tính được trong hai phút bằng giấy bút.

Nhưng ở đây phải phức tạp hoá một chút, vì cắt gọn sẽ thành sai. Với ResNet-18 ở tầng conv cuối, con số lý thuyết rơi vào cỡ vài trăm pixel, lớn hơn ảnh 224×224 , nên thoát nhìn có vẻ không có vấn đề gì. Hiệu chỉnh quan trọng là **trường tiếp nhận hiệu dụng** (*effective receptive field*) nhỏ hơn nhiều: đóng góp của các pixel giảm dần theo dạng gần Gaussian tính từ tâm, và bán kính hiệu dụng chỉ tăng cỡ căn bậc hai của số tầng. Đây là kết quả đã được công bố (dòng công trình về *effective receptive field*), và hệ quả thực dụng là hãy coi con số lý thuyết như một chặn trên rất lỏng chứ đừng coi là mô tả hành vi thật.

1.3 Bớt cách nhìn xa hơn, bớt cái giá (Widening the Receptive Field)

Khi receptive field không đủ, có bốn đòn bẩy và không cái nào miễn phí.

Bảng 1.1. Bốn cách mở rộng receptive field: cơ chế, cái được, và chế độ hồng riêng của từng cách.

Cách	Cơ chế	Được	Hồng khi nào
Tăng độ sâu	Cộng dồn $(k - 1)j$ qua nhiều tầng	Cả tầm nhìn lẫn độ trừu tượng	Khó tối ưu nếu không có residual; độ trễ tăng <i>tuần tự</i> , không song song hoá được
Lấy mẫu xuống	Nhân bước nhảy j lên	Rẻ nhất, mở rộng theo cấp số nhân	Mất chi tiết không gian vĩnh viễn; vật nhỏ biến mất trước khi tới tầng sâu
<i>Dilated conv</i> [2]	Giãn kernel mà giữ nguyên độ phân giải	Vừa xa vừa chi tiết — đúng thứ phân vùng cần	<i>Gridding artifact</i> : điểm lấy mẫu không phủ liên tục nên có tần số bị bỏ sót
Attention toàn cục [3]	Mọi vị trí chạm mọi vị trí trong một bước	Quan hệ xa tùy ý, do nội dung quyết định	Chi phí bậc hai theo số <i>token</i> ; bỏ bớt inductive bias nên đối dữ liệu

Cột chi phí của attention đáng làm thành con số vì nó giải thích gần hết thiết kế của các ViT hiện đại. Ở 224×224 với patch 16×16 ta có $14 \times 14 = 196$ token và ma trận tương tự $196^2 \approx 3,8 \cdot 10^4$ phần tử — hoàn toàn chấp nhận được. Nhưng phân vùng cần đặc trưng ở 56×56 , tức 3136 token, và $3136^2 \approx 9,8 \cdot 10^6$: gấp 256 lần. Đó là lý do *vật lý*, không phải thẩm mỹ, khiến Swin chia attention thành cửa sổ cục bộ rồi dịch cửa sổ giữa các tầng [4]. Nói cách khác, Swin *cài lại* tính cục bộ mà ViT vừa gỡ bỏ, sau khi cả ngành đã học được rằng gỡ hết thì quá đắt.

↔ **ràng buộc bởi** B/11 — con số $256 \times$ này vẫn chưa phải toàn bộ câu chuyện: thời gian chạy phụ thuộc lưu lượng bộ nhớ chứ không chỉ số phép tính.

1.4 Residual và luồng gradient (Residual Connections and Gradient Flow)

A. Nghịch lý mà residual thực sự giải quyết

Đây là điểm hay bị hiểu sai nhất trong toàn chương. Trước ResNet, người ta quan sát một nghịch lý: mạng 56 tầng có sai số *huấn luyện* cao hơn mạng 20 tầng [5]. Đây không thể là *overfitting* — overfitting làm sai số test cao chứ không làm sai số train cao. Mạng sâu hơn về nguyên tắc biểu diễn được mọi thứ mạng nông biểu diễn được (chỉ cần các tầng thừa học hàm đồng nhất), nên vấn đề nằm ở chỗ SGD *không tìm được* nghiệm đó. Kết nối tắt $y = x + F(x)$ sửa đúng chỗ đó: hàm đồng nhất trở thành mặc định (chỉ cần $F \rightarrow 0$), và gradient có một đường về thẳng không bị nhân qua chuỗi Jacobian. Nói gọn thì *residual giải quyết bài toán tối ưu trước*, còn *độ sâu chỉ là hệ quả của việc bài toán tối ưu đã dễ đi*.

B. Chuẩn hoá đặt ở đâu, và nó giả định gì

Hệ quả kéo dài tới hôm nay: mọi kiến trúc sâu đều có dạng “một luồng chính cộng dồn, các khối chỉ ghi thêm một lượng nhỏ vào luồng đó”. Chỗ đặt *normalization* là chi tiết đi kèm và có hệ quả rất thực. Đặt sau phép cộng (*post-norm*) thì luồng chính bị chuẩn hoá lại ở mỗi tầng, đường về của gradient không còn sạch, và *transformer* sâu cần khởi động học suất rất cẩn thận mới không phân kỳ; đặt trước khối (*pre-norm*) thì luồng chính không bị đung vào, huấn luyện ổn định hơn

nhiều nhưng thường mất một chút chất lượng cuối. Bản thân họ chuẩn hoá cũng là những giả định khác nhau.

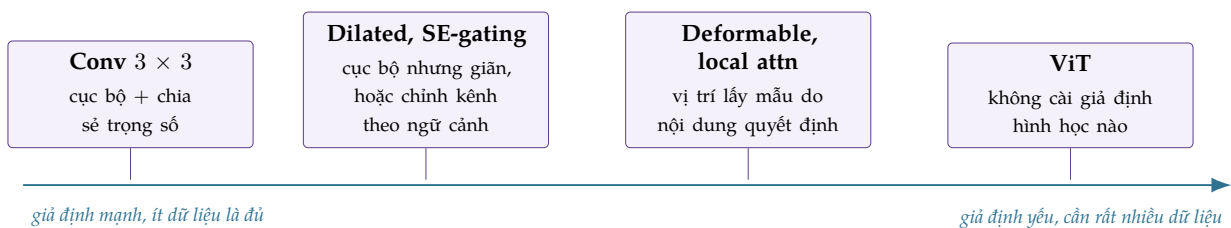
Bảng 1.2. Ba họ chuẩn hoá: mỗi họ là một giả định khác nhau về chỗ nào thống kê đáng tin.

Họ	Chuẩn hoá theo chiều nào	Giả định	Hỏng khi nào
BatchNorm [6]	Trên cả batch, từng kênh	Batch đủ lớn và đồng nhất; thống kê lúc suy luận giống lúc huấn luyện	Batch bằng 2 (cấu hình huấn luyện detection); dữ liệu suy luận lệch phân bố; suy luận trực tuyến từng mẫu
LayerNorm [7]	Trong từng mẫu, trên toàn bộ đặc trưng	Các kênh trong một mẫu có thang so sánh được	Ảnh có kênh với ý nghĩa vật lý rất khác nhau
GroupNorm [8]	Trong từng mẫu, theo nhóm kênh	Có cấu trúc nhóm tự nhiên giữa các kênh	Số nhóm chọn tùy tiện; mất một phần lợi ích chính quy hoá của BN

Ở đây cần trung thực về mức độ chắc chắn: lý do *vì sao* chuẩn hoá giúp vẫn còn tranh luận. Lời giải thích “giảm internal covariate shift” của bài gốc đã bị phản bác bằng thực nghiệm [9], và cách hiểu được ủng hộ hơn hiện nay là nó làm mất loss trơn hơn. Điều *đã được đo* là hành vi ở cột cuối bảng; điều còn là giả thuyết là cơ chế.

1.5 Định tuyến cố định hay theo nội dung (Fixed vs Content-Based Routing)

Convolution trộn thông tin theo một mẫu hình *cố định*: cùng một kernel áp lên mọi vị trí, bất kể nội dung. Đây vừa là sức mạnh (ít tham số, tổng quát hoá tốt) vừa là giới hạn cứng. Ba thể hệ giải pháp nới lỏng dần sự cố định đó, và xếp chúng lên một quang phổ như Hình 1.2 là cách gọn nhất để nhớ cả họ.



Hình 1.2. Quang phổ giả định của kiến trúc thị giác: đi từ trái sang phải, mỗi bậc gỡ bớt một ràng buộc hình học và đổi lấy nhu cầu dữ liệu cao hơn. Đây chính là trục của chương 4, lần này áp cho kiến trúc thay vì cho toàn hệ thống.

Ba bậc giữa và bậc cuối, đọc theo cái được và cái mất:

- **Channel gating** kiểu Squeeze-and-Excitation chỉ học một trọng số cho mỗi kênh dựa trên ngữ cảnh toàn cục [10]. Rất rẻ, và nó chứng minh rằng chỉ một chút định tuyến động đã đủ tạo khác biệt.

- **Deformable convolution** để mạng học luôn *vị trí lấy mẫu* thay vì cố định lưới [11]. Mạnh khi vật thể biến dạng nhiều; khó tối ưu hơn và khó tăng tốc trên phần cứng vì truy cập bộ nhớ không đều.
- **Self-attention** tính lại toàn bộ mẫu hình tròn cho từng đầu vào một. Cái mất không chỉ là FLOPs mà còn là *lượng dữ liệu tối thiểu*: ViT huấn luyện từ đầu trên ImageNet-1k thua CNN, và chỉ vượt lên khi có tiền huấn luyện quy mô lớn hoặc khi được thêm lại một ít inductive bias qua chưng cất và augmentation mạnh [12], [13].

Có một hướng ngược lại đáng nhắc: thay vì nói lỏng giả định, *cài thêm toán học vào kiến trúc* — tăng tối ưu khả vi, vòng lặp tối ưu được trải phẳng, ràng buộc hình học cứng đặt thẳng vào đồ thị tính toán. Tất cả đều nói “đừng học lại thứ tôi đã biết”. Chúng mạnh khi dữ liệu ít và mô hình toán đúng; chúng cứng nhắc và sai một cách khó gỡ khi giả định toán không khớp. Người làm hình học sẽ thấy trực này quen thuộc: đây đúng là cuộc tranh luận giữa pipeline SLAM cổ điển và các mô hình **tiến thẳng** (*feed-forward*), và chương 16 sẽ mở nó ra đầy đủ.

↪ **đối lập với** C/16 — hình học cổ điển cài toán vào hệ thống, mô hình tiến thẳng để mạng học lại toán đó — cùng một bài toán, hai vị trí đặt giả định.

1.6 Bốn thước đo chi phí (Four Cost Metrics)

A. Ba trục phân tích được bằng giấy bút

Có bốn thước đo và chúng *không* tỉ lệ với nhau: số tham số, FLOPs, bộ nhớ, và độ trễ thực đo. Trước khi chạy bất cứ thứ gì, ba trục kiến trúc cơ bản đã phân tích được.

Bảng 1.3. Nhân đôi theo ba trục: chi phí không đi cùng nhau, nên “cùng ngân sách FLOPs” không có nghĩa là “cùng giá”.

Nhân đôi	Tham số	FLOPs	Bộ nhớ kích hoạt	Độ trễ (batch 1)
Độ sâu	×2	×2	×2	×2 và <i>tuần tự</i> — không giấu được bằng song song hoá
Độ rộng	×4	×4	×2	< ×4: kernel to hơn dùng GPU hiệu quả hơn
Độ phân giải	×1	×4	×4	≈ ×4; là trục dễ tràn bộ nhớ nhất

Bảng 1.3 giải thích vài điều thoạt nhìn nghịch lý. Tăng độ rộng làm tham số tăng *bậc hai* — vì một tầng conv có $k^2 C_{in} C_{out}$ trọng số và cả hai chỉ số cùng nhân đôi — trong khi tăng độ sâu chỉ tăng tuyến tính; xét theo tham số thì sâu rẻ hơn rộng. Nhưng xét theo độ trễ ở batch 1 thì ngược lại: các tầng phải chạy nối tiếp, còn độ rộng thì trải ra được trên hàng nghìn lõi. Đây là lý do các mạng thiết kế cho thiết bị biên thường nông và rộng hơn ta tưởng. Và vì ba trục không tương đương, dồn hết ngân sách vào một trục là lựa chọn tồi. Mở rộng cả ba cùng lúc theo một tỉ lệ cố định — **compound scaling** — cho kết quả tốt hơn [14].

Điểm nối trực tiếp sang chương 11 nằm ở cột cuối: *FLOPs thấp không có nghĩa là nhanh*. MobileNet dùng convolution tách theo chiều sâu để giảm FLOPs hơn một bậc so với ResNet-50 [15]. Nhưng ở batch 1 trên GPU nó thường không nhanh hơn tương ứng, đôi khi còn chậm hơn. Lý do là mỗi FLOP của nó vẫn phải kéo theo gần như cùng lượng dữ liệu như trước, nên phép tính chuyển từ chế độ nghẽn tính toán sang chế độ nghẽn băng thông. Chương 11 biến câu đó thành hai con số byte cụ thể.

↔ **hệ quả của D/20** — mọi kỹ thuật nén mô hình ở đó đều nhằm vào một trong bốn thước đo này, và chọn nhầm thước đo là lý do phổ biến nhất khiến nén xong không nhanh hơn.

B. Các mô típ lặp lại: cách đọc nhanh nhất một kiến trúc lạ

- **Tách ba tầng chức năng** — *backbone* trích đặc trưng đa tỉ lệ, *neck* trộn các tỉ lệ (ví dụ FPN [16]), *head* biến đặc trưng thành đầu ra của bài toán. Nhận ra ba tầng này là bước đầu tiên khi mở một repo lạ, vì nó cho biết chỗ nào thay được.
- **Encoder-decoder với skip connection** [17] — dùng khi đầu ra cần độ phân giải cao: nhánh xuống lấy ngữ nghĩa, skip mang chi tiết không gian đã bị mất khi lấy mẫu xuống.
- **Bottleneck** — giảm số kênh ở giữa để cắt chi phí bậc hai của tầng 3×3 , rồi phục hồi.
- **Đa tỉ lệ và coarse-to-fine** — xử lý thô trước rồi tinh dần, để chi phí tập trung vào chỗ cần.

Chín mươi phần trăm kiến trúc thị giác là các mô típ này lặp lại theo thứ tự khác nhau, và khi bạn thấy chúng thì phần còn lại của bài báo đọc rất nhanh.

1.7 Thực hành

Repo và tài nguyên. [huggingface/pytorch-image-models](https://github.com/huggingface/pytorch-image-models) (timm) là nơi đối chiếu hàng trăm backbone trên cùng một khung; [TylerYep/torchinfo](https://github.com/TylerYep/torchinfo) để in bảng hình dạng và tham số từng tầng; [facebookresearch/fvcore](https://github.com/facebookresearch/fvcore) để đếm FLOPs; [lutzroeder/netron](https://github.com/lutzroeder/netron) để nhìn đồ thị tính toán thật thay vì tin hình vẽ trong paper; [Fangyh09/pytorch-receptive-field](https://github.com/Fangyh09/pytorch-receptive-field) để đối chiếu với phép tính tay của bạn. Danh sách này chốt tại tháng 9/2026 và tên các mô hình dẫn đầu sẽ cũ trong 6–12 tháng, còn các repo hạ tầng kể trên bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Tính tay receptive field của ResNet-18 ở tầng cuối — nó có phủ hết ảnh 224×224 không, và nếu có thì vì sao mô hình vẫn hỏng trên vật rất lớn? Vì sao tăng độ sâu rẻ hơn tăng độ rộng xét theo tham số nhưng đắt hơn xét theo độ trễ ở batch 1? Với batch 32 ảnh 512×512 , tầng nào chiếm nhiều bộ nhớ kích hoạt nhất, và vì sao câu trả lời không phải là tầng có nhiều tham số nhất?

Mini project. Chọn ba backbone trong timm có cùng mức accuracy ImageNet nhưng khác họ (một CNN cổ điển, một mạng *depthwise*, một ViT nhỏ). Tính tay tham số và FLOPs, *viết ra dự đoán thứ tự độ trễ trước khi đo*, rồi đo thật trên GPU và trên CPU. Mọi chỗ dự đoán sai là một bài học của chương 11; ghi lại cụ thể sai bao nhiêu lần và vì sao.

1.8 Giới hạn và trường hợp hỏng

Khung “kiến trúc là tập giả định” rất mạnh nhưng có ba chỗ nó nói dối.

- **Nó giả vờ rằng ta biết mình đang cài giả định gì.** Trong thực tế nhiều lựa chọn kiến trúc là kết quả của tìm kiếm thực nghiệm rồi được kể lại như một lập luận có nguyên lý. Bằng chứng rõ nhất là ConvNeXt [18]: khi lấy một ResNet và áp lần lượt các chi tiết huấn luyện lẫn thiết kế của ViT, phần lớn khoảng cách vốn được quy cho “attention” biến mất.
- **Mọi phép đếm ở đây là chặn, không phải hành vi thật.** Receptive field hiệu dụng nhỏ hơn lý thuyết rất nhiều, và FLOPs không dự đoán được độ trễ. Ai tối ưu theo con số lý thuyết mà không đo sẽ tối ưu nhầm hướng — chương 11 tồn tại đúng vì lý do này.
- **“Giả định mạnh thì cần ít dữ liệu” chỉ đúng khi giả định khớp.** Ảnh y sinh, ảnh vệ tinh và camera gắn trên người đều có xoay tự do. Với chúng, đẳng biến tịnh tiến của conv không

giúp gì cả, nên mạng phải tiêu sức chứa để học lại phép xoay — thứ mà một biểu diễn phù hợp hơn cho không mất gì. Cùng logic, nếu ảnh đã được cắt căn giữa và luôn cùng tỉ lệ thì phần lớn lợi thế của conv so với MLP biến mất.

CẠM BẤY

Đừng so sánh hai kiến trúc khi công thức huấn luyện của chúng khác nhau — và chúng gần như luôn khác nhau. Số epoch, augmentation, optimizer, độ phân giải, EMA của trọng số, độ dài warm-up: mỗi thứ có thể tạo chênh lệch lớn hơn chênh lệch giữa hai backbone. Khi đọc một bảng kết quả, câu hỏi đầu tiên không phải “kiến trúc nào cao hơn” mà là “hai dòng này có được huấn luyện như nhau không”; câu trả lời nằm ở phụ lục, không nằm ở phần method.

CẦN NHỚ

Kiến trúc không làm mô hình “thông minh hơn”, nó quyết định *ai chạm được tới ai, sau bao nhiêu bước, và với chi phí nào*. Ba câu hỏi tương ứng — receptive field, cách định tuyến, bốn thước đo chi phí — đủ để đọc gần như mọi kiến trúc thị giác, kể cả kiến trúc chưa ra đời.

[T2]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Inductive bias	Tập giả định về lời giải được cài sẵn trước khi nhìn dữ liệu. Nó đến từ bốn chỗ: kiến trúc, hàm mất mát, augmentation, và thuật toán tối ưu. Bạn không thể “không có giả định”, chỉ có thể chọn giả định nào.
●● Template matching	Cách đọc bộ phân loại tuyến tính: mỗi hàng của ma trận trọng số là một ảnh mẫu, và điểm số là tích vô hướng với ảnh đầu vào. Vẽ các hàng đó ra sẽ thấy mẫu “ngựa” có hai đầu, vì một lớp chỉ có một mẫu chuẩn.
●●● Vì sao không dùng một MLP khổng lồ cho ảnh?	Định lý xấp xỉ phổ quát chỉ nói hàm đúng <i>tồn tại</i> , không nói bạn tìm được nó. Một tầng nối đầy đủ từ ảnh $224 \times 224 \times 3$ sang 1000 nơ-ron đã là 150 triệu tham số, và mô hình vẫn không biết hai pixel cạnh nhau thì liên quan.
●●● Đăng biến tịnh tiến	Dời vật trong ảnh thì bản đồ đặc trưng dời theo đúng chừng ấy. Convolution có tính chất này gần như miễn phí nhờ chia sẻ trọng số, nhưng <i>không</i> có tính chất tương ứng cho xoay hay đổi tỉ lệ.
●●● Trường tiếp nhận	Vùng ảnh đầu vào ảnh hưởng tới một vị trí đầu ra. Tính bằng hai công thức truy hồi: bước nhảy nhân với stride, bán kính cộng thêm $(k - 1)$ lần bước nhảy hiện tại. Nhỏ hơn vật thể là một lớp lồi tính được trước khi huấn luyện.
●● Trường tiếp nhận hiệu dụng	Vùng thực sự đóng góp, nhỏ hơn con số lý thuyết rất nhiều vì đóng góp giảm dần theo dạng gần Gaussian từ tâm, và chỉ tăng cỡ căn bậc hai của số tầng. Con số lý thuyết chỉ là chặn trên rất lỏng.
●● Dilated convolution	Giãn khoảng cách giữa các điểm lấy mẫu của kernel để mở rộng tầm nhìn mà không giảm độ phân giải. Đúng thứ phân vùng cần, nhưng các điểm lấy mẫu không phủ liên tục nên sinh gridding artifact.
●● Vì sao attention đắt ở bài toán dày đặc?	Chi phí bậc hai theo số token. Ở 224×224 với patch 16×16 chỉ có 196 token, nhưng đặc trưng 56×56 cần 3136 token, đắt hơn 256 lần. Đó là lý do vật lý khiến Swin cắt attention về từng cửa sổ.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Kết nối tắt	Cộng thẳng đầu vào vào đầu ra của một khối, khiến hàm đồng nhất thành mặc định và mở một đường về sạch cho gradient. Nó giải quyết bài toán <i>tối ưu</i> trước; độ sâu chỉ là hệ quả.
●●● Vì sao mạng sâu hơn lại có sai số huấn luyện cao hơn?	Quan sát này có thật trước ResNet: mạng 56 tầng thua mạng 20 tầng ngay trên tập huấn luyện. Vì SGD không tìm được nghiệm, chứ không phải vì mạng không biểu diễn nổi. Đây chắc chắn không phải overfitting: overfitting làm sai số <i>kiểm thử</i> cao, không làm sai số huấn luyện cao.
●● Ba họ chuẩn hoá	BatchNorm chuẩn hoá theo chiều batch nên giả định batch đủ lớn và đồng nhất. LayerNorm và GroupNorm chuẩn hoá trong từng mẫu nên bỏ được giả định đó. Vì sao chuẩn hoá giúp thì tới nay vẫn còn tranh luận.
● Squeeze-and-Excitation	Học một trọng số cho mỗi kênh dựa trên ngữ cảnh toàn ảnh, rồi nhân lại vào kênh đó. Rất rẻ, và nó cho thấy chỉ một chút định tuyến theo nội dung đã đủ tạo khác biệt.
● Deformable convolution	Để mạng học luôn <i>vị trí lấy mẫu</i> thay vì cố định trên lưới. Mạnh khi vật thể biến dạng nhiều, nhưng truy cập bộ nhớ không đều nên khó tăng tốc trên phần cứng.
●●● Thước đo chi phí	Có bốn cái: số tham số, FLOPs, bộ nhớ, và độ trễ thực đo. Chúng không tỉ lệ với nhau, nên “cùng ngân sách FLOPs” không có nghĩa là “cùng giá”.
●● Vì sao nên mở rộng cả ba trục cùng lúc?	Vì ba trục có giá khác nhau: độ rộng làm tham số tăng bậc hai, độ sâu tăng tuyến tính nhưng cộng thẳng vào độ trễ tuần tự, độ phân giải nhân bộ nhớ lên bốn khi tăng gấp đôi. Dồn hết vào một trục thì chậm trễ của trục đó trước.
●● Backbone–neck–head	Ba tầng chức năng của gần như mọi hệ thống thị giác: trích đặc trưng đa tỉ lệ, trộn các tỉ lệ, rồi biến đặc trưng thành đầu ra của bài toán. Nhận ra ba tầng này là bước đầu tiên khi mở một repo lạ.
●● Mô hình sai có hệ thống trên vật rất lớn — nguyên nhân?	Hai khả năng: trường tiếp nhận hiệu dụng không phủ hết vật, hoặc tập huấn luyện thiếu vật cỡ đó. Phân biệt bằng cách cắt vật lớn thành mảnh nhỏ rồi đo lại: nếu điểm số hồi phục thì lỗi nằm ở tầm nhìn, không ở dữ liệu.
●● Huấn luyện detection với batch 2, kết quả tụt bất thường — nguyên nhân?	Nhiều khả năng là BatchNorm: thống kê tính trên hai mẫu thì quá nhiều, và thống kê chạy dừng lúc suy luận lệch hẳn. Đổi sang GroupNorm hoặc đóng băng BatchNorm rồi đo lại.
● Bảng nói kiến trúc mới hơn 3 điểm mAP — kiểm tra gì trước?	Kiểm tra phụ lục xem hai dòng có cùng công thức huấn luyện không: số epoch, augmentation, optimizer, độ phân giải, độ dài warm-up. ConvNeXt cho thấy phần lớn khoảng cách CNN–ViT biến mất khi giữ công thức cố định.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
timm	Nó tồn tại chính vì bài học “phải giữ công thức huấn luyện cố định mới so được kiến trúc”	Hạ tầng bền; là nơi duy nhất so backbone công bằng
ConvNeXt	Bằng chứng thực nghiệm trực tiếp rằng phần lớn khoảng cách CNN–ViT đến từ công thức huấn luyện chứ không từ attention	Kết quả này thay đổi cách đọc mọi bảng so sánh kiến trúc
Dòng Swin và các ViT phân cấp	Cài lại tính cục bộ và cấu trúc đa tỉ lệ vào ViT, đúng vì lý do bậc hai theo số token đã tính trong chương	Là kiến trúc mặc định cho bài toán dày đặc tính tới 9/2026

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Segment Anything (SAM)	Phân bổ chi phí cực lệch: bộ mã hoá ảnh rất nặng chạy một lần, còn phần giải mã mặt nạ rất nhẹ chạy nhiều lần	Minh hoạ sạch nhất cho việc tách backbone-head theo ràng buộc độ trễ
Họ EfficientNet và các biến thể sau nó	Mở rộng hợp nhất theo cả ba trục thay vì dồn ngân sách vào một trục	Ý tưởng còn sống dù tên mô hình đã cũ

MINI PROJECT SLAM · Đo lường tiếp nhận thực nghiệm của backbone frontend · vài giờ tới một ngày

Mục tiêu	kiểm chứng bằng số cái chặn trên lòng đã nêu trong chương: đặt con số trường tiếp nhận lý thuyết tính bằng giấy bút cạnh con số đo được bằng cách che ảnh, và xem chúng lệch nhau bao nhiêu lần.
Dữ liệu	vài trăm ảnh xám 752×480 từ một chuỗi EuRoC, cùng đúng backbone bạn định dùng cho frontend.
Các bước	(1) tính tay trường tiếp nhận lý thuyết ở ba tầng có độ sâu khác nhau bằng hai công thức truy hồi trong chương; (2) chọn một vị trí đầu ra cố định; (3) trượt một ô che 16×16 khắp ảnh và mỗi lần ghi lại mức thay đổi của vectơ đặc trưng tại vị trí đó; (4) vẽ bản đồ nhạy cảm và lấy bán kính chứa 90% tổng thay đổi; (5) lặp cho cả ba tầng.
Xong khi	bạn có một bảng ba dòng — tầng, trường tiếp nhận lý thuyết, bán kính 90% đo được — cùng tỉ số giữa hai cột, và trả lời được bằng số rằng backbone này có nhìn đủ xa cho vật thể lớn nhất xuất hiện trong chuỗi của bạn hay không.
Nổi với	B/11 vì bản đồ nhạy cảm tốn hàng nghìn lượt xuôi và là bài tập tốt để đo thông lượng; C/13 nếu bạn muốn lặp lại phép đo trên một ViT để so hai họ trên cùng thước đo; E/25 vì đây chính là một dạng bản đồ nổi bật.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì định lý chỉ nói hàm đúng *tồn tại* trong không gian giả thuyết, không nói bạn tìm được nó với lượng dữ liệu và tính toán bạn có. Một tầng nổi đầy đủ từ ảnh $224 \times 224 \times 3$ sang 1000 nơ-ron đã là 150 triệu tham số, và mô hình vẫn không biết hai pixel cạnh nhau thì liên quan.
- A2. Vì overfitting làm sai số *test* cao chứ không làm sai số *train* cao. Mạng sâu về nguyên tắc biểu diễn được mọi thứ mạng nông biểu diễn được, nên cái hỏng là bài toán tối ưu: SGD không tìm được nghiệm đó. Kết nối tắt biến hàm đồng nhất thành mặc định và mở một đường về sạch cho gradient.
- A3. Vì chi phí bậc hai theo số token. Ở 224×224 với patch 16×16 chỉ có 196 token, nhưng bài toán dày đặc cần đặc trưng ở 56×56 , tức 3136 token và đắt hơn 256 lần. Swin cài lại tính cục bộ mà ViT vừa gỡ bỏ, sau khi đã biết rằng gỡ hết thì quá đắt.
- A4. Vì tham số của một tầng conv là $k^2 C_{in} C_{out}$, nên nhân đôi độ rộng nhân tham số lên bốn còn nhân đôi độ sâu chỉ nhân lên hai. Nhưng các tầng phải chạy nối tiếp, còn các kênh trải ra được trên hàng nghìn lõi — nên ở batch 1, độ sâu là thứ không giấu được.
- A5. Vì hai dòng trong bảng gần như luôn khác nhau ở số epoch, augmentation, optimizer, độ phân giải và độ dài warm-up — những thứ có thể tạo chênh lệch lớn hơn chênh lệch kiến trúc. ConvNeXt là bằng chứng: giữ công thức cố định thì phần lớn khoảng cách biến mất.

NÓI LẠI THẬT ĐƠN GIẢN

Có hai cách xây nhà. Cách thứ nhất là đổ bê tông tự do: tạo được bất kỳ hình dạng nào, nhưng tốn rất nhiều vật liệu và thời gian. Cách thứ hai là lắp bằng một bộ gạch có sẵn, mỗi viên chỉ khớp với viên bên cạnh theo vài kiểu nhất định. Bộ gạch giới hạn hình dạng, nhưng bạn xây nhanh hơn với ít vật liệu hơn hẳn.

Mạng nơ-ron cũng vậy. Nối mọi điểm ảnh với mọi thứ là bê tông tự do: chỉ một tầng đã tốn hàng trăm triệu con số cần chỉnh. Hoặc bạn cài sẵn một hiểu biết rất tầm thường — hai điểm ảnh cạnh

nhau thì liên quan tới nhau, và một hoa văn có nghĩa ở góc này thì cũng có nghĩa y hệt ở góc kia. Cài xong, số thứ phải học giảm hàng nghìn lần.

Cái giá là gì? Vì mỗi viên gạch chỉ nhìn thấy vùng nhỏ quanh nó, muốn hai đầu bức ảnh “nói chuyện” được với nhau thì phải xếp rất nhiều lớp gạch chồng lên. Xếp không đủ lớp thì hai đầu đó không bao giờ gặp nhau, dù bạn dạy máy bao lâu — và đây là con số tính được bằng giấy bút trước khi bắt đầu. Còn nếu ảnh của bạn bị xoay lung tung thì bộ gạch không giúp gì, bạn tốn công hơn cả khi dùng bê tông.

Chỗ không rút gọn được: vì sao việc chuẩn hoá các con số giữa các lớp lại làm việc học dễ đi thì tới nay vẫn chưa có lời giải thích vừa gọn vừa đúng; cách giải thích trong bài báo gốc đã bị chính thực nghiệm bác bỏ.

Một câu: Kiến trúc là bộ gạch bạn chọn: chọn khớp với dữ liệu thì xây nhanh với rất ít vật liệu, chọn lệch thì tốn hơn cả xây tự do.

1.9 Kết nối

Chương này là *trục của chương 4 áp cho kiến trúc*: quang phổ ở Hình 1.2 chính là trục “giả định mạnh + ít dữ liệu ↔ giả định yếu + nhiều dữ liệu”. Nó *đối tác* với chương 9: kiến trúc cài giả định về dạng của hàm, loss cài giả định về mục tiêu. Nó là *tiền đề trực tiếp* của chương 11, vì cột “độ trễ” trong mọi bảng ở đây chỉ có nghĩa khi bạn hiểu vì sao FLOPs không dự đoán được thời gian.

Sang Phần C, C/13-kien-truc-backbone đọc lịch sử CNN, attention và ViT như một chuỗi các lần gỡ nút thắt — đó là bản kể chi tiết của quang phổ trong chương này. C/14-bai-toan-cv-loi cho thấy tách backbone/neck/head hoạt động ra sao trên bài toán thật và vì sao đa tỉ lệ là bắt buộc chứ không phải trang trí. C/16-thi-giac-3d mở ra cuộc tranh luận “nhúng toán vào kiến trúc hay để mạng học”. Và D/20-toi-uu-va-nen-mo-hinh nhật lại chỗ này khi bàn về việc thay đổi kiến trúc *sau* huấn luyện.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao định lý xấp xỉ phổ quát *không* phải lý do để dùng MLP cho ảnh — giới hạn thật nằm ở khả năng biểu diễn hay ở chi phí học?
2. Residual connection giải quyết vấn đề gì *trước tiên*, và bằng chứng thực nghiệm nào cho thấy đó không phải vấn đề overfitting?
3. Một mô hình phân vùng hồng có hệ thống trên vật rất lớn: hai chẩn đoán nào giải thích được triệu chứng đó, và phép đo nào phân biệt chúng?
4. Vì sao Swin phải chia attention thành cửa sổ, và phép tính nào về số token cho thấy đó là bắt buộc chứ không phải một lựa chọn thẩm mỹ?
5. Nếu dữ liệu của bạn có xoay tự do, đẳng biến tịnh tiến của convolution còn giá trị gì, và bạn sẽ cài giả định thiếu vào chỗ nào trong bốn chỗ của chương 4?
6. Hai mạng cùng số tham số, một sâu gấp đôi và một rộng gấp đôi: chênh lệch độ trễ ở batch 1 khác chênh lệch ở batch 64 ra sao, và vì sao?
7. BatchNorm hồng ở ba tình huống nào, và mỗi tình huống vi phạm giả định nào của nó?

Nguồn

- [1] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [2] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [3] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [4] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [5] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [6] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [7] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [8] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [9] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [10] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [11] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [12] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [13] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [14] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [15] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [16] T.-Y. Lin, P. Dollár, R. Girshick, et al., “Feature Pyramid Networks for Object Detection,” in *CVPR*, 2017.
- [17] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.
- [18] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.